Отчёт по лабораторной работе №3

Обработка изображений на двух GPU: MPI + CUDA

Выполнил: Кардаш Руслан, группа 25224

1 Задание

То же самое, что в задании 2, но с использованием двух видеокарт.

Напомним постановку: считать изображение из файла, отправить его на CUDA-устройство, применить фильтр — размытие, выделение границ (оператор Собеля) или медианный фильтр, — вернуть результат на хост и сохранить в PNG. Измерить накладные расходы и ускорение относительно CPU. Теория фильтров, операция свёртки и базовая CUDA-реализация подробно описаны в отчёте по заданию 2; здесь мы сосредоточимся на том, что добавляется при переходе к двум GPU.

2 Архитектура решения на двух GPU

2.1 Размещение GPU

Две GPU физически находятся на разных узлах кластера НОЦ «Газпромнефть-НГУ»: `gpn-node16` и `gpn-node17` (партиция `compclass`), — по одной NVIDIA Quadro RTX 4000 на узел. Узлы соединены сетью 10 Гбит/с. Задача запускается как один MPI-джоб с двумя рангами, по одному на ноду. Каждый ранг работает с локальной GPU узла через `cudaSetDevice(0)`.

2.2 Декомпозиция изображения

Изображение высотой H делится на N_{rank} горизонтальных полос равной высоты: ранг r обрабатывает строки $y \in [y_0, y_1)$, где

$$y_0 = \left\lfloor \frac{H\,r}{N_{\text{rank}}} \right\rfloor, \qquad y_1 = \left\lfloor \frac{H\,(r+1)}{N_{\text{rank}}} \right\rfloor.$$

Свёртка 3×3 требует соседних строк, поэтому на каждую ноду копируется срез с двумя дополнительными строками («гало»): $y \in [\max(0, y_0 - 1), \min(H, y_1 + 1))$. Это гарантирует корректную свёртку на стыках полос без дополнительного обмена строками между рангами. Граничные условия зажимаются через `ci(x,n)`.

2.3 Обмен данными

Схема работы:

1. Ранг 0 читает PNG или генерирует синтетическое изображение, затем рассылает его через `MPI_Bcast`.
2. Каждый ранг копирует свой срез с гало на свою GPU (`cudaMemcpy` H2D).
3. Ядро свёртки обрабатывает только строки своего ранга.
4. Результат копируется обратно на хост (`cudaMemcpy` D2H).
5. Ранг 0 собирает полный результат через `MPI_Gatherv` и сохраняет в PNG.

2.4 Особенности запуска на кластере

Партиция `gpuserv` содержит одну ноду и для двух GPU не подходит, поэтому используется `compclass` (8 нод, `MaxNodes=4`). GRES в Slurm не сконфигурирован (`GRES=(null)`), так что директива `--gres=gpu:1` не применяется: она вызывает ошибку, а GPU всё равно видна автоматически через CUDA API на каждом узле. OpenMPI на кластере собран без поддержки Slurm-PMI, поэтому запуск через `srun` падает с ошибкой в `pmix3x_client.c`. Обход — старт через `mpirun` с hostfile, в который записан список узлов задания (переменная `$SLURM_JOB_NODELIST`):

```
HOSTS=$(mktemp)
scontrol show hostnames $SLURM_JOB_NODELIST > "$HOSTS"
mpirun --hostfile "$HOSTS" -np 2 --map-by ppr:1:node \
       ./task3_filter input.png
```

Ключ `--map-by ppr:1:node` даёт ровно один процесс на ноду, то есть ровно одну GPU на процесс.

3 Численные результаты

Эксперимент проводился на двух узлах партиции `compclass` (`gpn-node16`, `gpn-node17`): по одной NVIDIA Quadro RTX 4000 на узел, 16 ядер CPU, 80 ГБ ОЗУ, Oracle Linux 8, CUDA Toolkit 12.3. Изображение — синтетическое, 4096×4096, 8-битное grayscale, генерируется самой программой. Время усреднялось по 10 запускам после прогрева; для ядра использовались события CUDA, для CPU — `std::chrono`. Корректность подтверждается столбцом *diff* — числом пикселей, отличающихся от CPU-результата более чем на 1.

Таблица 1. Результаты для двух GPU (по одному рангу на ноду), изображение 4096×4096.

Фильтр	T_{CPU}, мс	T_{kernel}, мс	T_{GPU}, мс	T_{overhead}, мс	S_{kernel}, раз	S_{total}, раз	diff, пикс.
Blur	144.021	0.334	2.134	1.800	431.7	67.5	0
Sobel	311.286	0.335	2.043	1.708	929.9	152.4	0
Median	431.046	2.767	4.483	1.716	155.8	96.1	0

Значения T_{kernel} и T_{GPU} измерены на одном ранге и представляют собой время обработки своего среза; оба ранга работают параллельно, поэтому полное время обработки всего изображения равно этим значениям.

3.1 Сравнение с одним GPU

Сопоставим полученные времена с результатами задания 2, где то же изображение обрабатывалось одной видеокартой.

Таблица 2. Сравнение с одним GPU (то же изображение 4096×4096).

Фильтр	T_{kernel}, мс		T_{GPU}, мс		Ускорение ядра
	1 GPU	2 GPU	1 GPU	2 GPU	
Blur	0.732	0.334	10.122	2.134	$2.19\times$
Sobel	0.754	0.335	9.501	2.043	$2.25\times$
Median	8.225	2.767	16.799	4.483	$2.97\times$

Выводы:

- Время ядра на ранг уменьшилось в 2.2–3.0 раза. Для blur и sobel это близко к линейному масштабированию: каждый ранг обрабатывает половину изображения, и время падает примерно вдвое. Для median ускорение даже выше двух — $2.97\times$: на уменьшенном рабочем наборе лучше работает L2-кэш GPU, а сортировка окна 3×3 сильно чувствительна к латентности памяти.
- Полное время T_{GPU} упало в 3.7–4.7 раза, что больше двух, — не только потому, что на каждой ноде вдвое меньше вычислений, но и потому, что пересылки H2D/D2H вдвое короче и эффективнее на байт: фиксированная составляющая латентности PCIe амортизируется на меньшем объёме.
- Накладные расходы $T_{\text{overhead}} = T_{\text{GPU}} - T_{\text{kernel}}$ на ранг составляют ≈ 1.7–1.8 мс — почти в 5 раз меньше, чем в однопроцессном случае (8.6–9.4 мс). Для 4096×4096 они заметно меньше времени CPU-обработки, что и даёт итоговое ускорение $S_{\text{total}} = 68$–$152\times$.
- Итоговое ускорение относительно CPU выросло с 19–$46\times$ (одна GPU) до 68–$152\times$ (две GPU). Наибольший прирост — у sobel: $45.8 \to 152.4\times$, так как у этого фильтра высокая арифметическая интенсивность на пиксель, и он лучше масштабируется с числом потоков.
- Корректность сохраняется: diff $= 0$ для всех фильтров. Граничные условия через `ci` и копирование гало-строк обеспечивают бесшовную обработку на стыках полос.

Замечание. Отметим, что T_{CPU} в задании 2 и задании 3 измерялись на разных узлах (`gpuserv` и `gpn-node16`), поэтому абсолютные значения различаются (например, для blur — 194 мс и 144 мс соответственно). Сравнение производительности GPU между 1 и 2 картами поэтому проводится по непосредственным T_{kernel} и T_{GPU}, а не по отношению S_{kernel} из разных прогонов.

3.2 Когда две GPU оправданы

Для 512×512 данные, измеренные в задании 2, показали, что накладные расходы одной GPU уже сравнимы с CPU-временем (≈ 4 мс против 3–9 мс). При переходе к двум GPU ситуация для малых изображений не улучшится: фиксированная стоимость (запуск MPI, инициализация CUDA, синхронизации, `MPI_Gatherv`) останется той же, а выигрыш от параллелизма будет съеден ею полностью. Оценка точки окупаемости для перехода $1 \to 2$ GPU лежит там же, где и для $0 \to 1$ GPU, — порядка 10^5–10^6 пикселей, и даже несколько выше из-за добавившихся коммуникаций.

Для 4096×4096 накладные расходы уже на порядок меньше CPU-времени, поэтому именно на больших изображениях две GPU дают максимальный эффект.

4 Общий вывод

Задание 3 — переход от одной GPU к двум — выполнено с использованием MPI и горизонтальной декомпозиции изображения. Программа автоматически масштабируется на любое число рангов; для $N = 2$ получены:

- ускорение ядра 2.2–$3.0\times$ — близко к линейному для линейных фильтров и сверхлинейное для median за счёт кэш-эффектов;
- ускорение полного конвейера H2D / kernel / D2H в 3.7–4.7 раза относительно $N = 1$;
- нулевое расхождение с CPU-эталоном во всех случаях;
- рост итогового ускорения по сравнению с CPU с 19–46 до 68–152 раз.

Тем самым продемонстрировано практическое применение многократных GPU-вычислений для задач обработки изображений и показано, что на кластере с одной GPU на узел основные ограничения ускорения — это фиксированная латентность пересылок, инициализации CUDA-контекста и операций редукции (`MPI_Gatherv`), а не вычислительная мощность самих GPU. Для больших изображений эти накладные расходы перестают доминировать, и две GPU дают ускорение, близкое к теоретическому максимуму.